Project: To-Do List API **CR ID:** CR-001 — Optimistic Locking

---

1. Technical Objective

Prevent lost updates on `Task` records when two clients read-then-write the same row concurrently, using JPA/Hibernate's built-in optimistic locking support rather than any custom locking mechanism.

2. Proposed Solution

Add a `@Version` field to the `Task` entity. Hibernate will automatically include the version in the `WHERE` clause of every `UPDATE` statement it generates for `Task`, and will throw `ObjectOptimisticLockingFailureException` if zero rows are affected (i.e. the row's version no longer matches). This is translated to a `409 Conflict` at the API boundary, consistent with how the project already maps `CategoryHasTasksException` and `DuplicateCategoryNameException` to `409` in `GlobalExceptionHandler`.

The client must supply the version it last read when submitting an update, so `TaskRequest` (or a small extension used only by the update path) must carry an `expectedVersion` field, and `TaskServiceImpl.update()` must set the managed entity's version to that value before Hibernate performs the update — otherwise Hibernate would simply use whatever version is already attached to the loaded entity, defeating the check.

3. Components to Modify

Component	File	Change
Task entity	`entity/Task.java`	Add `@Version private Long version;`
Task response DTO	`dto/TaskResponse.java`	Add `version` field
Task request DTO	`dto/TaskRequest.java`	Add `expectedVersion` field (required only on update; see §5)
Task service	`service/impl/TaskServiceImpl.java`	`update()` must verify/apply `expectedVersion` before saving
Error handling	`exception/GlobalExceptionHandler.java`	Add handler for `ObjectOptimisticLockingFailureException` → `409`

New components: none — this CR is implemented entirely within existing classes; no new entity, table, or endpoint is required.

4. Database Changes (JPA/Hibernate)

- Add to `Task`:

```
@Version
@Column(name = "VERSION", nullable = false)
private Long version;
```

- With `spring.jpa.hibernate.ddl-auto: update` (current setting), Hibernate will add the `VERSION` column to the existing `TASK` table automatically on next startup.
- **Migration consideration:** Hibernate's auto-update does not reliably backfill a `NOT NULL` column with a default value on every database/driver combination. For Oracle specifically, this should be verified in a non-production environment before relying on `ddl-auto: update` for this change; if existing rows end up with `NULL` versions, every pre-existing task would fail its first optimistic-lock check. The safer path is an explicit one-time `ALTER TABLE TASK ADD VERSION NUMBER(19,0) DEFAULT 0 NOT NULL` run once, after which Hibernate's version column mapping takes over for all subsequent writes. This is flagged as a migration risk, not assumed to be handled automatically.
- No new tables or constraints are required.

5. API Design

No new endpoints. Existing endpoints change as follows:

- `POST /api/tasks` — response now includes `version` (always `0` or `1` for a newly created row, depending on Hibernate's initial-version convention).
- `GET /api/tasks/{id}`, `GET /api/tasks` — responses include `version`.
- `PUT /api/tasks/{id}` — request body must include `expectedVersion`.

Example — successful update

Request:

```
PUT /api/tasks/42
{
  "title": "Prepare report",
  "status": "IN_PROGRESS",
  "categoryId": 3,
  "expectedVersion": 1
}
```

Response `200 OK`:

```
{
  "id": 42,
  "title": "Prepare report",
  "status": "IN_PROGRESS",
  "categoryId": 3,
  "categoryName": "Work",
  "version": 2
}
```

Example — stale version rejected

Request (same task, but `expectedVersion` is now out of date):

```
PUT /api/tasks/42
{
  "title": "Prepare report",
  "dueDate": "2026-11-01",
  "categoryId": 3,
  "expectedVersion": 1
}
```

Response `409 Conflict`:

```
{
  "timestamp": "2026-10-15T10:22:31Z",
  "status": 409,
  "error": "Conflict",
  "message": "Task 42 was modified by another request. Reload the task and retry.",
  "path": "/api/tasks/42"
}
```

6. Processing Logic

1. `TaskController.update(id, request)` receives the request, including `expectedVersion`.
2. `TaskServiceImpl.update()` loads the current `Task` via `taskRepository.findById(id)` (404 if missing, unchanged from today).
3. The service sets the loaded entity's fields from the request, including setting its `version` to `request.getExpectedVersion()` before the repository call (so Hibernate's dirty-check compares the client's claimed version against the database's actual stored version).
4. `taskRepository.save(task)` is called inside the existing transactional boundary. Hibernate issues `UPDATE TASK SET ...`

`, VERSION = VERSION + 1 WHERE ID = ? AND VERSION = ?`.

5. If the `WHERE` clause matches zero rows (version mismatch), Hibernate throws `ObjectOptimisticLockingFailureException` on flush.
6. `GlobalExceptionHandler` catches it and returns `409 Conflict` with a message naming the task and instructing the client to reload.
7. If the update succeeds, the response DTO is built with the new version (`oldVersion + 1`) and returned as `200 OK`, unchanged from today's flow otherwise.

7. Error Handling

Scenario	HTTP Status	Notes
`expectedVersion` missing from request	`400 Bad Request`	Validated like any other required field (consistent with existing `@NotNull` pattern)
`expectedVersion` does not match current version	`409 Conflict`	New `ObjectOptimisticLockingFailureException` handler
Task not found	`404 Not Found`	Unchanged — existing `ResourceNotFoundException` path
Two requests submit the exact same stale version simultaneously	One `200`, one `409`	Hibernate's row-level `WHERE` check guarantees only one wins; no application-level locking needed

8. Test Scenarios

See `TEST_CASES.md` for the full matrix. Summary: successful update advances version by 1; stale sequential update is rejected with `409`; two genuinely concurrent updates against the same task resolve to exactly one success and one conflict; all version-returning endpoints include the field; existing non-version-aware test scenarios are unaffected.

9. Technical Decision

Selected approach: JPA `@Version`-based optimistic locking (Hibernate native support).

Reason for selection: Requires no new infrastructure, no new table, and no polling. It is the standard, idiomatic way to solve lost-update problems in a Spring Data JPA application, and it integrates directly with the existing `GlobalExceptionHandler` error-mapping pattern already used in this project.

Alternatives considered:

- **Pessimistic locking (`SELECT ... FOR UPDATE`):** rejected — holds database locks for the duration of a user's edit session (or requires a separate lock/unlock API), which doesn't fit a stateless REST API and risks lock contention under normal multi-user load.
- **Last-write-wins (no concurrency control, current behavior):** rejected — this is the problem being solved.
- **Timestamp-based (`updatedAt` comparison) instead of a numeric version:** rejected — offers no real advantage over `@Version` here, and loses the guarantee that every write strictly increments a comparable counter (clock skew / same-millisecond writes are a risk with timestamps).

10. Technical Approval

- **Prepared By:** Backend Developer
- **Reviewed By:** Tech Lead
- **Approval Status:** Pending
- **Implementation Status:** Not Started